CHECK POINT

AI SECURITY STACK MAPPER

Configuration & Operating Instructions

Step-by-step: configure, run & operate the platform

v1.0.0

June 2026

Follow the parts in order: **A. Configure** the platform once, then **B. Operate** it day to day, and use **C. Maintenance** for upkeep. Commands run from the project root unless a step says otherwise.

Part A · Configuration (one-time setup)

A1. Check prerequisites

- **Node.js 20+** and npm; **Docker + Docker Compose** for the container path.
- *(Optional)* **Ollama** or an OpenAI-compatible runtime for live model evaluation.

Verify: `node -v` prints v20+; `docker --version` succeeds.

A2. Choose a deployment mode

MODE	USE WHEN	URL
Docker Compose	Realistic demo / shared env (Postgres)	http://localhost:8080
Full stack (dev)	Local development (SQLite)	http://localhost:5173
Frontend only	Quick offline preview (no login)	http://localhost:5173

A3. Configure the backend environment

```
cd server
cp .env.example .env
```

Edit `server/.env` — minimum for a real deployment:

```
DATABASE_URL="postgresql://user:pass@host:5432/stackmapper"   # or file:./dev.db
PORT=4000
CORS_ORIGIN="https://your-frontend-origin"
JWT_SECRET="<a long random string>"
NODE_ENV=production
# Vendor keys (leave blank to stay in mock mode):
CHECKPOINT_TE_API_KEY=
LAKERA_GUARD_API_KEY=
```

Note: with `NODE_ENV=production`, demo accounts are NOT seeded — you create the admin in A5.

A4. Initialize the database

```
npm install
npm run setup        # prisma generate + migrate deploy + seed (dev)
# production (no demo data): npx prisma generate && npx prisma migrate deploy
```

Verify: migrations apply with no error; (dev) seed prints "Seeded: …".

A5. Create the first administrator

```
npm run create:admin -- --email you@org.com --name "You" --password 'StrongPass!23'
```

Expected: "Created owner you@org.com." In dev you may instead use the demo Owner `owner@imbsys.com` / `Demo!1234`.

A6. Start the services

Docker (recommended):

```
docker compose up --build      # → http://localhost:8080
```

Or full stack (dev):

```
cd server && npm start         # API on :4000
# new terminal:
npm install && npm run dev      # UI on :5173
```

A7. Verify the platform is up

```
curl http://localhost:4000/api/health      # {"ok":true,...}
```

Expected: opening the app shows the login screen and the header reads **Backend online**.

A8. (Optional) Connect vendor integrations & local LLM

1. Set `CHECKPOINT_*` / `LAKERA_*` keys in `.env` and restart the API.
2. On the **Integrations** page the connector should switch from *mock* to **connected**.
3. For local model evaluation: `ollama pull llama3.2:1b`.

Part B · Operating the Application

B1. Sign in

Open the app, enter your credentials (or a demo account), and click **Sign in**.

Expected: the Dashboard loads; your name + role appear top-right.

B2. Set your preferences

Settings → Preferences: choose language, default project, notifications. These save to your account.

B3. Register your LLM models (AppSec/Owner)

1. **Model Coverage → Register model**.
2. Set provider, deployment (hosted/local/custom), endpoint, environment, sensitivity, owner, use case.
3. Toggle the guardrails that apply (input/output/tool/logging).

Expected: the model appears as a card with a computed risk level.

B4. Test & evaluate a model

1. Click **Test** — confirms the endpoint is reachable (latency).

2. Click **Evaluate** — runs adversarial probes (injection, jailbreak, leakage, harmful, PII).

Expected: a resilience score (e.g., "100% resilient") with each probe's verdict and the model's real response. Unreachable endpoints return a *simulated* result.

B5. Scan a file before it enters RAG (SOC/AppSec/Owner)

1. **File Security Gateway** → pick source + scan engine.
2. Drop a file. Read the verdict; malicious/suspicious are blocked from retrieval.

B6. Inspect a prompt (SOC/AppSec/Owner)

Lakera Guard → choose Input/Output, paste a prompt, **Send**. Review the risk score, decision, and violations; tune thresholds in the Policy panel.

B7. Assess controls & review risks

1. **Layer Assessment** — mark each control Not implemented → Planned → Implemented → Verified.
2. **Risk Register** — update risk status; expand to see mitigating controls.

Expected: the Dashboard AI Security Score updates as posture changes.

B8. Check compliance & generate reports

1. **Compliance** — review coverage across OWASP, NIST AI RMF, EU AI Act, ISO 42001, MITRE ATLAS; **Export report** per framework.
2. **Report Generator** — export the full assessment as Markdown or PDF.

B9. Administer users (Owner)

Settings → **User Management**: **Add user**, change roles inline, reset passwords, or delete. You can't delete yourself or the last Owner.

Part C · Maintenance & Operations

C1. Enforce the security-score gate (CI)

```
cd server
SCORE_MIN=60 SCORE_PROJECT=proj-copilot npm run score:gate
```

Result: exits non-zero (fails the pipeline) if the project's score is below the threshold.

C2. Back up & restore data

- **Postgres:** `pg_dump` / `pg_restore` the database; the Docker volume is `pgdata`.
- **SQLite (dev):** copy `server/prisma/dev.db`.

C3. Update / re-deploy

```
git pull
cd server && npm install && npx prisma migrate deploy   # apply new migrations
docker compose up --build -d                            # or restart the services
```

C4. Run tests & stop the stack

```
npm test && (cd server && npm test)      # verify before shipping
docker compose down                       # stop (add -v to drop the DB volume)
```